

Update 3 – Weekly Update Report

CSE 499B: Senior Design II | Section: 12 | Group: 03

Week Update Summary

In recent weeks, I enhanced a Python Selenium Automated Agent with an MCP StdIo Server to reliably submit restricted Google Forms using Chrome with a cloned Google profile. I also addressed a handful of stability issues, namely: singleton locking and extension cookie errors. Additionally, I prepared and processed the Software Questions.csv dataset, tokenized it using GPT-2 tokens, and converted it into a Hugging Face Dataset for fine-tuning the GPT-2 model. As an extension to that project, I built a prototype of an AI mock interview system using a locally-running open-source LLM from Google Colab, as well as modules to generate questions, evaluate answers with structured feedback, evaluate difficulty level through adaptive difficulty logic, and have developed an extremely basic Backend / Frontend user interface for user interaction.

Contribution

Md. Kamrul Islam (2211745642):

The prior week, I worked on enhancing an automation agent for Google Forms written in Python and Selenium, using an MCP stdio server to provide form-filling capabilities with a logged-in Google Account. The automation agent will now launch Chrome, using a clone of the user's Google profile to enable authenticated sessions. Additionally, I fixed numerous stability issues with the agent, including a SingletonLock issue caused by conflicting profiles and a cookie issue in an extension where a file was mistakenly created as a directory. These fixes now allow the automation agent to fill out and submit restricted Google Forms consistently.

Farhan Ishraque (2212002042):

I rode the waves of Software Questions.csv (dataset), created a text column combining Question and Answer for model input, and then tokenized our data with a GPT-2 tokenizer. I took special care to maintain data integrity by paying attention to sequence length and the pad token during the preprocessing. Finally, I converted the pandas DataFrame into a Hugging Face dataset. After tokenizing the dataset, I split the data into training (80%) and test (20%) sets for model validation. I fine-tuned a pre-trained GPT2 using the HuggingFace Trainer API, providing Training Arguments to set the batch size, learning rate, evaluation strategy, etc., to optimize training. Finally, I saved the fine-tuned model and tokenizer, allowing them to be reused for future inference. Then I read some research papers for the mock interview literature review.

Iram Shehzad (2131614642):

This week, I worked on creating the main prototype of the AI-based mock interview system utilising an open-source LLM that runs locally in Google Colab without the need for external

APIs. I put in place a question generation module that generates technical interview questions based on the chosen role, topic, and degree of difficulty, as well as an answer evaluation module where the model evaluates a candidate's response and produces structured feedback that includes strengths and weaknesses as well as scores for technical accuracy, clarity, and depth. In order to transform the model's output into structured data that the system can handle automatically, I also created a parser and a JSON-based output format. I used these extracted scores to create a straightforward adaptive decision logic that uses the candidate's performance to decide the next interview question's difficulty. At this point, the prototype effectively finishes the entire pipeline utilising a locally run LLM, from question formulation to answer evaluation and adaptive decision making.

Atikul Islam Nahid (2211978042):

This week, I focused on integrating my trained AI interview assistant with both a backend and a frontend interface. After fine-tuning the DeepSeek-R1 model using the LoRA technique, I connected it to a backend system that handles user input and communicates with the AI model. I am also trying to develop a simple front-end interface that allows a candidate to interact with the system during a mock interview. The frontend sends the candidate's answers to the backend, which then passes them to the AI model to generate the next interview question and evaluate the response. Additionally, I am testing the system with sample interview questions to analyze how effectively the model conducts the interview.

Completed Task:

- Various model training with the dataset.
- Read, write, and automatic Google Form submission.

Work in Progress

- Comparing various models' performance.
- Integrating a custom model into the mock interview feature.
- Upload files to a Google Form and submit it automatically.

Next Target

- Integrate the custom model into the mock interview feature.
- Fully read and write on a Google Form and submit it automatically.

Challenges

- Selecting and using the specific models for a specific task.
- Bypassing Google Chrome's automation blocking.
- Costly existing tools or websites in the current market.